Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

CE-1107 Fundamentos de Arquitectura de Computadores

Bitácora

Proyecto Grupal 1: Máquinas de estados

Sistema de puerta con contraseña

Profesor: Luis Chavarría Zamora

Integrantes:

Isaac Andrés Arias Picado
Byron Josué Bolaños Porras
Jose Andrés Chavarría Gamboa
Isac Marín Sirias

II Semestre 2026

Índice

1. Herramientas de automatización utilizadas **2**

2. Entradas **2**
18 de octubre de 2026 2
19 de octubre de 2026 3
21 de octubre de 2026 4
23 de octubre de 2026 6
24 de octubre de 2026 6
27 de octubre de 2026 8
28 de octubre de 2026 10
29 de octubre de 2026 11
30 de octubre de 2026 13
31 de octubre de 2026 14
1 de noviembre de 2026 15
2 de noviembre de 2026 16

1. Herramientas de automatización utilizadas

Herramienta	Uso en el proyecto
Quartus Prime Lite 25.1	Síntesis del diseño, asignación de pines de la DE10-Standard, análisis de tiempos y programación de la FPGA por JTAG.
Questa Altera Starter FPGA Edition 2025.2	Compilación y simulación de los módulos y de sus bancos de pruebas.
EDA Playground (Riviera-PRO)	Simulación de algunos módulos del bloque de salidas.
Git y GitHub	Control de versiones, ramas por tarea e integración mediante pull requests.
Visual Studio Code	Edición del código SystemVerilog y de la documentación.
Overleaf y LaTeX	Redacción de la bitácora.

2. Entradas

18 de octubre de 2026

Integrante(s): Isaac Andrés Arias Picado

Tareas realizadas

Se creó el repositorio con la propuesta de implementación, el proyecto de Quartus y el archivo `.gitignore`. En la rama `setup/inicial` se implementó el paquete `config_pkg`, que reúne todas las constantes del sistema (frecuencias, tiempos, códigos de estados y eventos) para que ningún otro módulo tenga números fijos. Este cambio se integró con el pull request #1. Después, en la rama del bloque A, se implementaron los módulos `clk_enables` y `debounce` con sus testbenches y el script `run.do` que corre todas las simulaciones.

Módulo clk_enables

Genera pulsos de un ciclo a partir del reloj de 50 MHz para que los demás módulos trabajen a frecuencias bajas sin dividir el reloj. Produce un pulso de captura cada 2 s y un pulso de 1 kHz que comparten el debounce y el display. Usar un solo contador para estos dos ahorra flip-flops. El reset entra por la habilitación del contador y no por una entrada asíncrona.

$$
\begin{aligned}
CAP_BITS = \lceil \log_2(100\,000\,000) \rceil = 27, \quad SHARED_BITS = \lceil \log_2(50\,000) \rceil = 16 \\
tick_capture = (cap_cnt = 99\,999\,999) \wedge \neg reset \\
tick_debounce = tick_display = (shared_cnt = 49\,999) \wedge \neg reset \\
en_{cap} = \neg reset \wedge \neg is_max_{cap} \\
cnt_{next} = en \wedge (cnt + 1)
\end{aligned}
$$

Módulo debounce

Los botones solo pasan por dos flip-flops de sincronización y un detector de flanco, ya que la FPGA tiene Schmitt trigger. El detector de dedos viene de hardware externo sin Schmitt trigger,

por lo que se filtra exigiendo 20 muestras iguales seguidas a 1 kHz. Luego se valida que haya como máximo un dedo activo y se codifica a un símbolo de 0 a 5.

$$
\begin{aligned}
finger_valid &= (stable_0 + stable_1 + stable_2 + stable_3 + stable_4 \leq 1) \\
sym_0 &= stable_0 \vee stable_2 \vee stable_4 \\
sym_1 &= stable_1 \vee stable_2 \\
sym_2 &= stable_3 \vee stable_4
\end{aligned}
$$

Cuadro 1: Filtro de estabilidad del detector de dedos (cada tick de 1 kHz)

Condición	**candidate**	**count**	**stable**
reset	00000	0	00000
raw $\neq$ candidate	raw	1	Sin cambio
raw = candidate y count $<$ 19	Sin cambio	count + 1	Sin cambio
raw = candidate y count = 19	Sin cambio	20	candidate

Verificación

El testbench de `clk_enables` pasó sus 4 pruebas (período de captura, período compartido, reset y primer tick). El de `debounce` pasó sus 9 pruebas, que cubren el pulso único de cada botón, el filtro de 20 muestras y la validación de patrones con dos o más dedos.

19 de octubre de 2026

Integrante(s): Isaac Andrés Arias Picado

Tareas realizadas

Se implementó el módulo `finger_capture` con su testbench, se documentó el bloque A y se integró a la rama `development` con el pull request #2.

Módulo finger_capture

Mantiene una ventana con los últimos 3 símbolos capturados, de 3 bits cada uno. En cada pulso de captura entra el símbolo nuevo y sale el más viejo. Los símbolos inválidos se ignoran, para que una lectura errónea no desplace un símbolo bueno.

$$
\begin{aligned}
clear_all &= reset \vee clear_capture \\
shift_en &= tick_capture \wedge finger_valid \wedge \neg clear_all \\
mask_hold &= \neg shift_en \wedge \neg clear_all \\
window_{next} &= (shift_en \wedge \{window[5:0], sym\}) \vee (mask_hold \wedge window)
\end{aligned}
$$

Cuadro 2: Ejemplo de llenado de la ventana

Tick	Símbolo	Válido	Ventana
1	001	1	000 000 001
2	010	1	000 001 010
3	011	1	001 010 011
4	100	1	010 011 100
5	100	0	010 011 100 (sin cambio)

Verificación

El testbench pasó sus 6 pruebas, que revisan el reset, la limpieza, el desplazamiento, que un símbolo inválido no cambie la ventana y el orden correcto tras cuatro capturas.

21 de octubre de 2026

Integrante(s): Isac Marín Sirias

Tareas realizadas

Se implementaron los cinco módulos del bloque D, encargado de las salidas del sistema, junto con sus testbenches y su documentación.

Módulo seven_seg

Convierte un código de 5 bits en los siete segmentos de un display, activos en bajo. Para cada carácter se genera un mintérmino con el AND de los cinco bits de entrada, y cada segmento es el NOT del OR de los mintérminos de los caracteres que lo encienden. Además de 0 a F, muestra las letras o, c, n y t para los estados y modos, y los códigos 20 a 31 dejan el display apagado.

$$character_1 = \neg c_4 \wedge \neg c_3 \wedge \neg c_2 \wedge \neg c_1 \wedge c_0$$
$$seg_a = \neg(character_0 \vee character_2 \vee character_3 \vee \ldots \vee character_{15})$$

Módulo pwm

Genera una señal de 1 kHz con un ciclo de trabajo de $s/7$. Un contador recorre los 50 000 ciclos de cada período, y la salida está en alto mientras el contador es menor que un umbral que depende de la velocidad. El umbral se elige con máscaras y la velocidad nueva solo se toma al empezar un período.

Cuadro 3: Umbral del PWM por velocidad

Velocidad	Ciclos en alto	Duty
0	0	0 %
1	7143	14.3 %
2	14286	28.6 %
3	21429	42.9 %
4	28571	57.1 %
5	35714	71.4 %
6	42857	85.7 %
7	50000	100 %

$$pwm_out = running \wedge \neg reset \wedge (counter < threshold)$$
$$threshold = \bigvee_{s=1}^{7} \big((active_speed = s) \wedge H(s)\big)$$

Módulo motor_ctrl

Es combinacional y dirige el PWM a una de las dos líneas del puente H según la dirección. Las dos salidas nunca pueden estar activas a la vez, porque una exige $dir_1 \wedge \neg dir_0$ y la otra $\neg dir_1 \wedge dir_0$.

$$line_{abrir} = \neg reset \wedge dir_1 \wedge \neg dir_0 \wedge pwm$$
$$line_{cerrar} = \neg reset \wedge \neg dir_1 \wedge dir_0 \wedge pwm$$

Módulo buzzer_ctrl

Genera una onda cuadrada para un buzzer pasivo. La frecuencia define el tono y un contador de duración define el patrón de cada evento. Los sonidos continuos se mantienen mientras el código esté activo, y los finitos se inician con un pulso y terminan solos. Un sonido continuo cancela uno finito.

Cuadro 4: Eventos del buzzer

Código	Evento	Frecuencia	Patrón
1	TEST	1000 Hz	Continuo
2	REG_OK	1000 Hz	Un beep de 100 ms
3	VALID	1000 Hz	Dos beeps de 100 ms
4	INVALID	500 Hz	Un beep de 500 ms
5	OPENING	2000 Hz	Continuo
6	CLOSING	1500 Hz	Continuo

Módulo display_ctrl

Selecciona el carácter de cada uno de los seis displays y usa una instancia de `seven_seg` por display, sin barrido. HEX0 muestra el estado de la puerta, HEX1 a HEX3 los tres dígitos capturados, HEX4 el modo y HEX5 la velocidad. En administración y prueba, HEX0 muestra la posición recordada de la puerta.

Verificación

Se prepararon testbenches para los cinco módulos. Cada uno compara las salidas contra una tabla de referencia independiente, por ejemplo las 32 entradas del decodificador o las 16 combinaciones del control del motor. Según la documentación del bloque, la ejecución de estas simulaciones queda pendiente.

23 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se revisó el estado general del repositorio y cada una de sus ramas ya hechas hasta el momento para comparar el enunciado con el archivo de propuesta en la rama principal. Se analizó cómo adecuar el bloque C (encargado de la máquina de estados principal e integración con el sistema) de trabajo a la implementación ya realizada en los bloques A y D, esto porque la máquina de estados principal debe poder conectarse correctamente a estas.

Procedimiento y decisiones

Al comparar el enunciado con el trabajo realizado se encontraron varios puntos importantes a resolver. En primer lugar se identificó que no hay forma de verificar cuándo una contraseña es inválida o no ya que la comparación se realiza de manera continua. Además, como el patrón de sin poner ningún dedo se establece como un cero, constantemente la ventana se llena de ceros sin requerir interacción con el sistema. Se plantea cambiar el sistema por intentos y un contador, así al completar el intento se evalúa y se limpia la captura.

Se realizó también una reestructuración del árbol de commits para ajustarse a lo que pide el enunciado (una nueva rama de trabajo por cada nueva implementación creada desde la rama development). Este cambio se integró mediante pull request y los cambios y fechas quedaron intactos.

24 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Módulo timer_travel

Se implementó el contador que usa la máquina de estados para medir tiempos. Cuenta milisegundos usando el tick de 1 kHz que ya genera el bloque A, así el registro necesita pocos bits. La primera versión tenía una salida de espera de 1 s y un timeout de 30 s, pensando en usar finales de carrera para detectar la llegada de la puerta. Más tarde se decidió detectar el fin de carrera por tiempo, por lo que el timeout se reemplazó por una tabla de tiempos de viaje según la velocidad.

El tiempo de viaje sigue la fórmula de la propuesta y se guarda en milisegundos en el paquete de configuración.

$$
\begin{aligned}
T(s) &= \frac{7 \cdot T_{max}}{s}, \quad T_{max} = 4000 \text{ ms} \\
&\Rightarrow T(1) = 28000,\ T(2) = 14000,\ \ldots,\ T(7) = 4000 \text{ ms} \\
W &= \lceil \log_2(28000 + 1) \rceil = \lceil 14.77 \rceil = 15 \text{ bits}
\end{aligned}
$$

La tabla se implementó con máscaras, comparando la velocidad con cada valor y dejando pasar solo el tiempo correspondiente, de la misma forma que el umbral del PWM. El contador se detiene en el viaje más largo para no desbordarse y para que la espera de 1 s siga funcionando cuando la velocidad es 0.

$$
\begin{aligned}
clr &= reset \lor start \\
inc &= tick_ms \land \neg clr \land \neg at_max \\
hold &= \neg clr \land \neg inc \\
count_{next} &= (inc \land (count + 1)) \lor (hold \land count) \\
limit &= \bigvee_{k=1}^{7} \big((speed = k) \land T(k)\big) \\
wait_done &= (count \geq 1000) \\
travel_done &= (count \geq limit)
\end{aligned}
$$

Módulo attempt_ctrl

Se creó un módulo nuevo que detecta cuándo termina un intento de contraseña. Sin él, la comparación sería continua y nunca habría un momento para decidir que una contraseña es inválida. El intento empieza con el primer dígito distinto de 0, ya que el 0 es lo que se captura cuando nadie pone los dedos, y termina al capturar el tercer dígito. La salida ready se registra un ciclo después, porque en ese mismo flanco la ventana del bloque A guarda el tercer dígito.

$$
\begin{aligned}
clr &= reset \lor clear_capture \\
shift &= tick_capture \land finger_valid \\
nz &= sym_2 \lor sym_1 \lor sym_0 \\
start &= shift \land nz \land (cnt = 0) \land \neg clr \\
adv &= shift \land (cnt = 1) \land \neg clr \\
fin &= shift \land (cnt = 2) \land \neg clr \\
hold &= \neg clr \land \neg start \land \neg adv \land \neg fin \\
cnt_{next} &= (start \land 01) \lor (adv \land 10) \lor (hold \land cnt) \\
ready_{next} &= fin
\end{aligned}
$$

Diseño de la máquina de estados

Se rediseñó la tabla de transiciones de la propuesta para que cumpla el enunciado. Las dos contraseñas guardadas pasan a ser equivalentes, cualquiera abre la puerta si está cerrada y la cierra si está abierta. En administración el submodo escoge el espacio de memoria, y si la contraseña ingresada ya existe se borra, si no se guarda. Se agregó el estado VALID, que espera 1 s antes de mover el motor, porque el sonido continuo de apertura cancelaría el sonido de contraseña válida. Además, el reset borra las contraseñas pero no detiene un movimiento en curso, para que la puerta no quede a medio camino y el sistema no pierda su posición.

Cuadro 5: Tabla de transiciones de la FSM principal

Estado	**Condición**	**Siguiente**	**Acción**
ERROR	rst	ERROR	Borrar contraseñas
ERROR	adm	ADMIN	mode $\leftarrow 1$
ERROR	tst y puerta cerrada	TEST	Fase 0
ERROR	ready	ERROR	INVALID
CLOSED/OPEN	rst	ERROR	Borrar contraseñas
CLOSED/OPEN	adm	ADMIN	mode $\leftarrow 1$
CLOSED	tst	TEST	Fase 0
CLOSED/OPEN	ready y match	VALID	Guardar velocidad
CLOSED/OPEN	ready y no match	Igual	INVALID
VALID	wait_done y speed $\neq 0$	OPENING/CLOSING	Arrancar motor
VALID	wait_done y speed $= 0$	CLOSED/OPEN	No se mueve
OPENING/CLOSING	rst	Igual	Borrar contraseñas, motor sigue
OPENING/CLOSING	travel_done	OPEN/CLOSED*	Motor apagado
ADMIN	rst	ERROR	Borrar contraseñas
ADMIN	mode $= 0$	CLOSED/OPEN*	Salir
ADMIN	adm	ADMIN	mode: $1 \rightarrow 2 \rightarrow 0$
ADMIN	ready y match	ADMIN	Borrar, REG_OK, mode $\leftarrow 0$
ADMIN	ready y no match	ADMIN	Guardar en espacio mode, REG_OK, mode $\leftarrow 0$
TEST	fase 0 y travel_done	TEST	Fase 1 (espera 1 s)
TEST	fase 1 y wait_done	TEST	Fase 2 (cerrar)
TEST	fase 2 y travel_done	CLOSED*	Fin

* Pasa a ERROR si no hay contraseñas registradas. Prioridad: rst > adm > tst > ready.

Verificación

El timer se verificó con un testbench de 14 pruebas, que revisa el tiempo de viaje de las siete velocidades justo en el tick límite, la saturación del contador, el caso de velocidad 0 y el reinicio con start y con reset. El módulo attempt_ctrl se verificó con 7 pruebas usando el módulo de captura real del bloque A, para confirmar que al llegar ready la ventana tiene los tres dígitos. Todas las pruebas pasaron.

27 de octubre de 2026

Integrante(s): Byron Josué Bolaños Porras

Tareas realizadas

Se implementaron los tres módulos del bloque B, encargado del cálculo de la velocidad y de la memoria de contraseñas, cada uno con su testbench y su documentación, en la rama `BloqueB-Datos`. El bloque se ajustó a las señales definidas para la máquina de estados principal, con dos contraseñas equivalentes y la operación de la ALU dada por el primer dígito de 1 a 5.

Módulo alu

Calcula la velocidad de la puerta con los tres dígitos de la ventana. El primer dígito elige la operación y los otros dos son los operandos A y B. El resultado es de 3 bits y circular, lo que da una velocidad de 0 a 7. No existe operación nula, ya que un intento nunca empieza con 0, y cualquier código fuera de 1 a 5 da 0. La multiplicación se arma con productos parciales sin usar el operador de multiplicar, y como girar 3 bits tres veces devuelve el mismo valor, las rotaciones usan B mód 3 giros.

Cuadro 6: Operaciones de la ALU

op	Operación	Resultado
1	Suma	$(A + B)$ mód 8
2	Resta	$(A - B)$ mód 8
3	Multiplicación	$(A \cdot B)$ mód 8
4	Rotación izquierda	$rot_{izq}(A,\ B \text{ mód } 3)$
5	Rotación derecha	$rot_{der}(A,\ B \text{ mód } 3)$

$$
\begin{aligned}
mult_0 &= A_0 \wedge B_0 \\
mult_1 &= (A_1 \wedge B_0) \oplus (A_0 \wedge B_1) \\
mult_2 &= (A_2 \wedge B_0) \oplus (A_1 \wedge B_1) \oplus (A_0 \wedge B_2) \oplus \big((A_1 \wedge B_0) \wedge (A_0 \wedge B_1)\big) \\
mod_1 &= (B = 1) \vee (B = 4) \vee (B = 7), \quad mod_2 = (B = 2) \vee (B = 5) \\
rotl &= (mod_0 \wedge A) \vee (mod_1 \wedge \{A_1 A_0 A_2\}) \vee (mod_2 \wedge \{A_0 A_2 A_1\}) \\
result &= (sel_{sum} \wedge sum) \vee (sel_{sub} \wedge sub) \vee (sel_{mult} \wedge mult) \vee (sel_{rotl} \wedge rotl) \vee (sel_{rotr} \wedge rotr)
\end{aligned}
$$

Módulo password_mem

Guarda hasta dos contraseñas en dos espacios equivalentes, cada uno con su bandera de validez. La máquina de estados escribe, borra o limpia mediante pulsos de un ciclo. En cada espacio el borrado tiene prioridad sobre la escritura, para que una escritura simultánea con un borrado nunca deje datos a medias.

Cuadro 7: Transición de cada espacio de memoria

kill	wr	pass siguiente	valid siguiente
1	X	000 000 000	0
0	1	window	1
0	0	Sin cambio	Sin cambio

$$
\begin{aligned}
kill_1 &= reset \vee clear_all \vee del_1 \\
load_1 &= wr_1 \wedge \neg kill_1 \\
hold_1 &= \neg wr_1 \wedge \neg kill_1 \\
pass1_{next} &= (load_1 \wedge window) \vee (hold_1 \wedge pass1) \\
valid1_{next} &= load_1 \vee (hold_1 \wedge valid1)
\end{aligned}
$$

Módulo comparator

Compara la ventana contra las dos contraseñas guardadas con un XOR bit a bit y un OR de reducción. Un espacio vacío nunca coincide, porque sin la bandera de validez una ventana en ceros igualaría una memoria vacía y daría un acierto falso.

$$equal_1 = \neg \bigvee_{i=0}^{8} (window_i \oplus pass1_i)$$
$$match_1 = equal_1 \wedge valid_1$$

Verificación

Los testbenches de la ALU, la memoria y el comparador pasaron sus 13, 15 y 11 pruebas. La ALU se probó con las 64 combinaciones de operandos para cada operación, la memoria con los casos de prioridad entre escritura y borrado, y el comparador con un barrido de las 512 ventanas posibles.

28 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se actualizó la propuesta del repositorio para que coincida con el rediseño de la máquina de estados y se integró a development mediante pull request. En el paquete de configuración se agregó el código del estado VALID y los submodos de administración. Luego se implementó el módulo fsm_main con su testbench y su documentación, y por último el módulo top, que une los cuatro bloques, junto con la asignación de pines de la tarjeta.

Módulo fsm_main

La máquina se construyó fila por fila a partir de la tabla de transiciones. Cada fila de la tabla se convierte en una señal que vale 1 cuando se está en ese estado y se cumple su condición. Como el enunciado no permite if, case ni ternario, la prioridad entre botones no se puede dar con un else, así que cada fila lleva negadas las condiciones completas de las filas de arriba de su mismo estado. Por ejemplo, en ERROR el botón de prueba solo cuenta si la puerta está cerrada, entonces la fila de ready niega esa condición entera y no solo el botón.

$$e_adm = is_error \wedge \neg rst \wedge adm$$
$$e_tst = is_error \wedge \neg rst \wedge \neg adm \wedge tst \wedge \neg door$$
$$e_rdy = is_error \wedge \neg rst \wedge \neg adm \wedge \neg(tst \wedge \neg door) \wedge ready$$

Cada registro se escribe con su propia expresión, sumando con OR el valor que carga cada fila enmascarado por su señal, más un término que mantiene el valor cuando ninguna fila lo toca. Las filas que van a ERROR no necesitan término en el siguiente estado porque su código es 000, basta con que apaguen el mantener. El submodo y la fase del test solo se mantienen dentro de su estado, de modo que fuera de ADMIN y de TEST vuelven solos a 0.

$$
\begin{aligned}
hold_{st} &= \neg(e_adm \vee e_tst \vee co_rst \vee co_adm \vee \ldots \vee t_end) \\
state_{next} &= \neg reset \wedge \big((e_adm \vee co_adm) \wedge S_{ADMIN} \vee \ldots \vee hold_{st} \wedge state\big) \\
speed_{next} &= \neg reset \wedge \big(co_ok \wedge alu_result \vee (e_tst \vee c_tst) \wedge TEST_SPEED \vee spd_{hold} \wedge speed\big) \\
door_{next} &= \neg reset \wedge \big(opg_done \vee (door \wedge \neg clg_done)\big)
\end{aligned}
$$

Al escribir el código se notó que la tabla de transiciones no mostraba todos los efectos de cada fila. La entrada a TEST, por ejemplo, también carga la velocidad del test y reinicia el timer, y la evaluación de un intento limpia la captura. Por esto la documentación del módulo se actualizó con una columna de acciones completa, junto con las ecuaciones de todas las señales.

Módulo top

El módulo top instancia los módulos de los cuatro bloques y los conecta entre sí, casi sin lógica propia. Incluye un reset de encendido con un contador de 4 bits que arranca en 0 al programar la FPGA y mantiene el reset activo durante 8 ciclos. Los botones KEY de la tarjeta son activos en bajo, por lo que entran negados. Además, durante VALID se le pasa al display el estado CLOSED u OPEN según la posición de la puerta, para que muestre la puerta y no la letra E de error.

Los pines se tomaron del manual de la DE10-Standard. Los sensores de dedos usan GPIO 0 a 4, las dos líneas del motor GPIO 5 y 6, y el buzzer GPIO 7. En el archivo de Quartus se cambió la entidad principal a top, ya que el nombre anterior empezaba con un número y no es válido para un módulo, y se quitó del proyecto un testbench que no debe sintetizarse. Para compilar se trajo a la rama el bloque B, integrándolo primero a development con su pull request.

Verificación

El testbench de fsm_main usa el timer_travel real y un modelo simple de la memoria del bloque B, con el tick de 1 ms acelerado. Tiene 16 pruebas con 30 comprobaciones, entre ellas el registro y borrado de contraseñas en los dos espacios, la apertura y el cierre, la velocidad 0, el reset durante un movimiento, la secuencia de prueba completa y la prioridad del reset sobre admin. Todas pasaron en Questa. Al revisar los módulos del grupo también se encontraron operadores ternarios en debounce.sv, lo que se le comunicó al encargado del bloque A junto con una versión con máscaras.

29 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se verificó el sistema completo en simulación con todos los bloques conectados mediante el módulo top, y se compiló por primera vez el proyecto en Quartus para la DE10-Standard. También se revisó de nuevo el enunciado contra lo implementado y se empezó a armar el circuito de los sensores de dedos en protoboard.

Verificación del sistema completo

Se escribió un testbench para el módulo top que usa los módulos reales de los cuatro bloques. Como la captura toma un dígito cada 2 s, simular tres intentos con los tiempos reales tardaría horas,

por lo que se creó una copia del paquete de configuración solo para simulación, donde la captura ocurre cada 2000 ciclos y el tick de 1 ms cada 10 ciclos. La copia conserva el mismo nombre de paquete, así que ningún módulo se modifica y en Quartus se sigue usando el paquete original.

La prueba registra una contraseña poniendo los dedos en los sensores, la usa para abrir la puerta revisando el PWM del motor y el display, prueba una contraseña incorrecta y termina con el reset. Las 9 comprobaciones pasaron en Questa.

Compilación en Quartus

La primera compilación terminó sin errores, pero entre las advertencias aparecieron dos problemas. El primero fue un desborde en las constantes de duración del buzzer, que se calculaban multiplicando los milisegundos por la frecuencia del reloj antes de dividir entre 1000. Ese producto no cabe en un entero de 32 bits, y la duración del sonido de contraseña inválida llegaba a dar un valor negativo.

$$500 \cdot 50\,000\,000 = 2.5 \times 10^{10} > 2^{31} - 1$$
$$\Rightarrow \text{se divide primero} \quad \frac{50\,000\,000}{1000} \cdot 500 = 25\,000\,000$$

El segundo fue el aviso de que no se cumplían los tiempos, porque sin un archivo de restricciones Quartus supone un reloj de 1 GHz. Para ambos se dejó preparada la corrección, que se aplicó al día siguiente.

Al revisar el enunciado se confirmó que el display debe mostrar la velocidad normalizada de 0 a F, como estaba en la propuesta, lo que requiere un cambio en el control del display del bloque D. Además se notó que el enunciado habla de símbolos de cero a cinco dedos, mientras que el diseño usa un solo dedo cuya posición indica el dígito, por lo que el punto quedó para discutirse con el grupo.

Circuito de los sensores de dedos

Cada sensor usa una fotorresistencia en un divisor de tensión. Como el divisor entrega un voltaje analógico y la FPGA espera niveles lógicos claros, cada salida pasa por un inversor con Schmitt trigger del integrado 74HC14, que tiene seis inversores y se alimenta con los 3.3 V de la tarjeta. La fotorresistencia va hacia 3.3 V y la resistencia fija hacia tierra, de modo que al tapar el sensor el nodo baja y el inversor entrega un 1.

$$V_{nodo} = 3.3 \cdot \frac{R}{R + R_{LDR}}$$
$$R \approx \sqrt{R_{luz} \cdot R_{dedo}} = \sqrt{2\ \text{k}\Omega \cdot 40\ \text{k}\Omega} \approx 8.9\ \text{k}\Omega$$

A 3.3 V el inversor cambia a 1 cuando la entrada baja de aproximadamente 1 V y vuelve a 0 cuando sube de 1.8 V. El cálculo da cerca de 9 kΩ con el dedo bien puesto, pero se escogió una resistencia menor, de 4.7 kΩ, para que el sensor siga funcionando aunque el dedo no tape del todo. Con ella el nodo queda en 2.3 V sin dedo, en 0.35 V con el dedo bien puesto y en 0.63 V si la fotorresistencia solo llega a 20 kΩ. La quinta fotorresistencia cambiaba muy poco entre luz y oscuridad, así que se decidió reemplazarla. La posición de los pines de alimentación del conector GPIO no aparece en el manual, por lo que se identificó midiendo con el multímetro antes de conectar.

Durante el armado una pata de una resistencia hizo contacto entre la línea de 3.3 V y tierra, y se notó por el olor y porque la pata cambió de color. Se desconectó de inmediato, se revisó que la

tarjeta siguiera funcionando, se cambió la resistencia y se cortaron las patas al ras para evitar otro corto.

30 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se aplicaron las correcciones encontradas en la primera compilación y se volvió a compilar el proyecto. Se terminó de armar en protoboard el circuito de los sensores con fotorresistencias nuevas, se diseñó el manejo del buzzer con un transistor y se conectaron los optoacopladores. Por último se cargó el diseño por primera vez en la FPGA.

Correcciones en Quartus

En el paquete de configuración se cambió el cálculo de las duraciones del buzzer para dividir primero entre 1000 y después multiplicar por los milisegundos, de modo que ningún valor intermedio supere el rango de un entero de 32 bits. Además se agregó un archivo de restricciones que indica que el reloj de 50 MHz tiene un período de 20 ns, y se agregaron al archivo de exclusiones de git los archivos que genera Quartus al compilar.

Con esto la compilación pasó de 38 a 8 advertencias, todas informativas, y desaparecieron el desborde y el aviso de tiempos. El análisis de tiempos reporta un margen de 9.45 ns en el peor caso, así que el camino más largo del diseño tarda cerca de 10.5 ns, dentro de los 20 ns del período. Durante una de las compilaciones se cambió de rama mientras Quartus todavía corría, y la última etapa leyó los archivos de la otra rama, por lo que se tomó como regla cerrar Quartus antes de cambiar de rama.

Sensores de dedos

Las fotorresistencias nuevas miden entre 3 y 3.5 kΩ con luz y entre 30 y 35 kΩ con el dedo encima, es decir, cambian unas diez veces, el doble que las anteriores. El valor ideal de la resistencia fija era de unos 6.8 kΩ, pero entre las disponibles se usó la de 10 kΩ. Con ella los cinco nodos quedaron cerca de 2.2 V sin dedo y el peor caso con dedo fue de 0.67 V, por lo que los cinco inversores cambiaban correctamente.

Al repetir la prueba en un cuarto con menos luz, las salidas se quedaban siempre en 1, como si siempre hubiera un dedo. Sin dedo el nodo medía 1.8 V, lo que corresponde a una fotorresistencia de unos 8.3 kΩ.

$$R_{LDR} = R \cdot \frac{3.3 - V_{nodo}}{V_{nodo}} = 10\ \text{k}\Omega \cdot \frac{3.3 - 1.8}{1.8} \approx 8.3\ \text{k}\Omega$$

Ese voltaje queda justo en el umbral de subida del Schmitt trigger, así que una vez que el dedo bajaba la salida a 1, el nodo ya no subía lo suficiente para regresarla a 0. Con esto se vio que la resistencia fija depende de la iluminación del lugar, porque con menos luz la fotorresistencia sube tanto sin dedo como con dedo. Como regla práctica se tomó una resistencia fija de entre 2 y 2.5 veces el valor de la fotorresistencia sin dedo, lo que deja el nodo cerca de 2.3 V. Para la defensa se decidió llevar resistencias de varios valores y medir las fotorresistencias en el aula antes de presentar.

Buzzer y optoacopladores

El buzzer resultó ser pasivo, ya que con voltaje constante solo hace un clic, y su bobina mide 14 Ω. Un pin de la FPGA no puede entregar esa corriente, por lo que se maneja con un transistor NPN 2N3904 que funciona como interruptor. El pin solo alimenta la base y la corriente del buzzer sale de la línea de 3.3 V del conector, que soporta hasta 1.5 A. En paralelo con el buzzer se colocó un diodo 1N4148 para absorber el pico que genera la bobina al apagarse.

Con 1 kΩ en la base el transistor no se saturaba y el colector quedaba en 1.1 V. Se bajó la resistencia de base a 470 Ω y se agregó una resistencia de 10 Ω en serie con el buzzer, con lo que el colector bajó a 0.107 V.

$$I_B = \frac{3.3 - 0.7}{470} \approx 5.5 \text{ mA}$$
$$I_C = \frac{3.3 - 0.1}{10 + 14} \approx 133 \text{ mA}$$

Los LED de los optoacopladores se conectaron a los pines de la FPGA con una resistencia de 330 Ω, y sus cátodos a la tierra de la parte lógica, separada de la tierra de la fuente del motor.

$$I_{LED} = \frac{3.3 - 1.2}{330} \approx 6.4 \text{ mA}$$

Primera prueba en la FPGA

Se cargó el diseño con el programador de Quartus por JTAG. Al iniciar, el display mostró la E de error y la n del modo normal, los botones de administración y prueba respondieron, y los dígitos de los sensores entraron en el display cada 2 s. El modo de prueba completó la secuencia de abrir, esperar y cerrar en unos 15 s, como se esperaba para la velocidad de prueba.

31 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se continuó el ensayo funcional del sistema en la FPGA y se corrigió un problema de visualización en el display de los dígitos ingresados. Además se revisaron los entregables que pide el enunciado y los datos físicos que hacen falta para la propuesta de investigación.

Visualización del último intento

Durante el ensayo los dígitos entraban correctamente desde los sensores, pero el tercer dígito de cada intento nunca llegaba a verse en el display. Al capturarse el tercer dígito, el control de intentos avisa a la máquina de estados, esta evalúa la contraseña y en el ciclo siguiente limpia la ventana de captura. Como esto ocurre en unos 20 ns, el display pasaba directamente a mostrar ceros y el usuario no podía comprobar lo que había ingresado.

Se consideraron dos soluciones. La primera, propuesta por un compañero, era retrasar la evaluación hasta la siguiente captura, 2 s después, para que el intento se viera antes de sonar el resultado.

La segunda era darle al display su propia copia de la ventana, actualizada solo cuando entra un dígito nuevo. Se escogió la segunda porque la respuesta del sistema sigue siendo inmediata y el cambio queda solo en el módulo top, sin tocar la máquina de estados ni el control de intentos.

La copia se carga un ciclo después de cada captura, cuando la ventana ya contiene el dígito nuevo, y se mantiene igual el resto del tiempo. Así, aunque la máquina de estados limpie la ventana, el display sigue mostrando el intento completo hasta la siguiente captura, y después vuelve a mostrar los dígitos que entran.

$$
\begin{aligned}
shift &= tick_capture \wedge finger_valid \\
shift_d_{next} &= shift \\
disp_{next} &= \neg reset \wedge \big((shift_d \wedge window) \vee (\neg shift_d \wedge disp)\big)
\end{aligned}
$$

Verificación

Al testbench del sistema completo se le agregó una prueba que ingresa un intento, revisa que la ventana quedó en cero después de la evaluación y que el display todavía muestra los tres dígitos. Pasaron las 10 comprobaciones en Questa. En Quartus la compilación terminó sin errores, con 559 celdas lógicas y un margen de 9.86 ns en el peor caso de tiempos.

Entregables y datos físicos

Según el enunciado, en la defensa se deben mostrar simulaciones junto con el diseño corriendo en la FPGA, además de un diagrama general del sistema desde el usuario hasta los módulos y un diagrama interno de la FPGA hecho por el grupo. Para la sección de datos de la propuesta de investigación se definió medir, para cada velocidad de 1 a 7, cinco tiempos de apertura y cinco de cierre, anotando si la puerta llega al final del recorrido. También se medirá el voltaje promedio del PWM en el pin de cada dirección, que debe ser cercano a $3.3 \cdot s/7$ V, y la corriente del motor con el multímetro en serie con su fuente.

1 de noviembre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se elaboró el diagrama general del sistema que pide el enunciado, desde el usuario hasta los actuadores.

Diagrama general del sistema

El diagrama sigue el recorrido de la información. El usuario coloca los dedos sobre las fotorresistencias, cuyos divisores pasan por el 74HC14 y entran a la FPGA. Dentro de la FPGA se muestran los cuatro bloques en el orden en que trabajan, entradas, datos, control y salidas. De la FPGA salen tres caminos, los displays de siete segmentos, el transistor que maneja el buzzer y los optoacopladores que controlan el puente H y el motor de la puerta. La alimentación del motor se dibujó como un grupo aparte para dejar claro que tiene su propia fuente y su propia tierra, separadas de la tarjeta.

Se hizo en Mermaid porque el diagrama se escribe como texto, de modo que puede guardarse en el repositorio, revisarse en los commits y modificarse sin redibujarlo. Desde el código se exporta la

imagen en PNG para la documentación. El diagrama interno de la FPGA, con los módulos y sus señales, queda para una entrada posterior.

2 de noviembre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se subió el diagrama general a la rama de documentación y se hicieron en la FPGA las mediciones físicas para la propuesta de investigación. Durante las pruebas se cambió el motor, se bajó la frecuencia del PWM, se ajustaron los tonos del buzzer y se corrigió el orden de los dígitos en el display. Después se tomaron fotografías del montaje, se actualizó la propuesta y se abrió el pull request del Bloque C hacia `development`. También se elaboró el diagrama interno de la FPGA y se revisó todo el código del repositorio para eliminar lo que no tenía uso y cerrar los pendientes del enunciado.

Condiciones de las pruebas

Las pruebas se hicieron en una habitación con buena iluminación y resistencias fijas de 10 kΩ en los sensores. El puente H se alimentó con una batería de 9 V a través de un regulador de 6 V. La batería midió 8.87 V al inicio y 8.53 V al final. Como no se cuenta con osciloscopio, todas las mediciones se hicieron con multímetro y cronómetro.

Voltaje promedio del PWM

Con el multímetro en la línea de apertura se midió el voltaje mientras la puerta abría a cada velocidad. El multímetro promedia la señal, así que la lectura debe ser cercana a $3.3 \cdot s/7$ V. Los valores fueron desde 0.477 V en $s = 1$ hasta 3.338 V en $s = 7$, todos alrededor de 1.1 % por encima de lo esperado. La diferencia se debe a que el nivel alto real del pin es 3.338 V y no 3.3 V, porque al dividir cada lectura entre la de $s = 7$ el ciclo de trabajo coincide con $s/7$ en menos de 0.1 puntos porcentuales. El voltaje sube 0.477 V por cada nivel con un ajuste lineal casi perfecto, por lo que las siete velocidades quedan repartidas de forma pareja.

Temporizador de recorrido

Se cronometró la duración del tono de apertura, que suena durante todo el recorrido programado, para cada velocidad. Los tiempos siguieron la relación $T(s) = 28/s$ s, desde 28.5 s en $s = 1$ hasta 4.21 s en $s = 7$. En promedio el tiempo medido fue 0.29 s mayor que el programado, y esa diferencia no creció con la duración, por lo que se atribuye al tiempo de reacción al usar el cronómetro y no al temporizador.

Motor y frecuencia del PWM

Con el motor original y el PWM a 1 kHz, la velocidad 1 no lograba arrancar el motor aunque el pin entregaba el voltaje promedio correcto, y a la velocidad 7 daba muchas vueltas durante los 4 s del recorrido. Se consultó al profesor, quien indicó que no importa la cantidad de vueltas siempre que se note el cambio de ciclo de trabajo, y sugirió revisar la frecuencia.

Se cambió el motor por un micromotorreductor 150:1 de 6 a 12 V, que gira cerca de 90 RPM a 6 V y da un movimiento más lento y con más fuerza. A 1 kHz este motor arrancaba desde la velocidad

2, pero casi no se notaba diferencia entre niveles. Se concluyó que los pulsos eran demasiado cortos para el optoacoplador y el puente H, que tardan en conmutar y deforman el pulso, y que sin carga el motor sigue girando por inercia durante el tiempo apagado.

Se bajó la frecuencia a 200 Hz y el motor arrancó en todas las velocidades, con diferencias visibles entre niveles. Luego se dejó en 100 Hz para tener más margen. Con esto el período del PWM pasa a ser de 500 000 ciclos de reloj, y el umbral de cada velocidad se multiplica por diez sin cambiar el ciclo de trabajo.

$$N = \frac{50\,000\,000}{100} = 500\,000 \text{ ciclos}$$
$$H(1) = \text{round}\left(\frac{500\,000 \cdot 1}{7}\right) = 71\,429 \text{ ciclos}$$

La corriente del motor nuevo a velocidad 7, sin carga, fue de 0.03 A, coherente con los cerca de 40 mA que indica el fabricante a 6 V. Vista de frente, la apertura gira en sentido horario y el cierre en sentido antihorario, y el motor se detiene al terminar cada recorrido.

Buzzer y display

El tono continuo de apertura y cierre, de 2000 Hz y 1500 Hz, resultaba muy fuerte durante todo el recorrido. Se bajó a 800 Hz para la apertura y 600 Hz para el cierre, con lo que el buzzer suena más grave y con menos volumen, y los dos movimientos se siguen distinguiendo.

En el display los tres dígitos del intento aparecían al revés, con el primero a la derecha. Se invirtió el orden de los grupos de tres bits al conectar la copia de la ventana con el control del display, de modo que el dígito más reciente quede en HEX1 y el intento se lea de izquierda a derecha.

Verificación y evidencia

Con todos los cambios se volvió a compilar en Quartus sin errores, con 566 celdas lógicas y un margen de 9.76 ns en el peor caso de tiempos. En la FPGA se recorrieron todos los estados y se fotografió el display en cada uno, error al encender, administración, intento aceptado, apertura, puerta abierta, cierre y prueba. También se fotografiaron el montaje completo, los sensores, el buzzer y la etapa de potencia.

Con estos datos se actualizó la propuesta de investigación, con la etapa física, las tablas de mediciones, la simulación del PWM a 100 Hz y el análisis del cambio de frecuencia.

Diagrama interno de la FPGA

El enunciado pide un diagrama general de la FPGA diseñado por el grupo, y aclara que no es válido el que genera el software. Se hizo en Mermaid a partir del módulo `top`, que define todo lo que se implementa en la tarjeta. Sus puertos son las entradas y salidas físicas, y sus instancias son los módulos del diseño. El diagrama agrupa los módulos en los cuatro bloques y marca en cada flecha las señales que conectan un módulo con otro. El reloj y el reset de encendido llegan a todos los módulos, por lo que se indicaron con una nota en lugar de una flecha a cada uno. El paquete de configuración no aparece como bloque, porque solo contiene constantes y no genera hardware.

Al revisar el diagrama contra `top.sv` se notó que la señal `tick_display` salía de `clk_enables` y no llegaba a ningún módulo. Se había planeado para refrescar displays multiplexados, pero la DE10-Standard tiene pines propios para cada HEX y el control del display quedó combinacional.

Revisión y limpieza del código

Se revisaron todos los archivos de `src` buscando señales, constantes e importaciones sin uso. Se eliminaron `tick_display`, las importaciones del paquete en cuatro módulos que no usan ninguna de sus constantes, y una serie de constantes del paquete que nadie usaba. Estas venían de la planificación inicial, cuando el paquete se escribió antes que los módulos. Durante la implementación cada módulo resolvió su parte de forma directa, por ejemplo con anchos fijos de 9 bits o con los códigos de caracteres escritos en el propio módulo, y esas constantes quedaron sin referencias.

El filtro de entradas tenía operadores ternarios, que el enunciado prohíbe. Un compañero lo reescribió con máscaras, y en la revisión se simplificaron además unos términos que siempre valían cero. Para comprobar que el comportamiento no cambió, se simuló la versión nueva junto con la original durante 300 000 ciclos con entradas aleatorias, y las salidas coincidieron en todos los ciclos.

Velocidad normalizada y orden de los dígitos

El enunciado pide mostrar la velocidad normalizada de 0 a F. En el control del display se agregó un multiplexor por máscaras que convierte cada velocidad en el valor más cercano de la escala hexadecimal.

$$V_{HEX} = \text{round}\left(\frac{15 \cdot s}{7}\right)$$

$$s = 1, 2, \ldots, 7 \Rightarrow V_{HEX} = 2,\ 4,\ 6,\ 9,\ \text{B},\ \text{D},\ \text{F}$$

También se movieron al control del display dos ajustes que estaban en el módulo superior. En el estado de validación ahora muestra la posición de la puerta, igual que en administración y prueba, y los dígitos del intento quedan en orden de lectura. Con esto el módulo superior le pasa el estado y la ventana directamente, sin lógica intermedia.

Verificación de la limpieza

Los dieciséis bancos de prueba terminaron sin errores. Se actualizaron el del generador de habilitaciones y el del display, este último con la tabla de velocidades normalizadas, y se renombró el banco del decodificador de siete segmentos para que coincida con el nombre de su módulo. Además se simuló el sistema completo antes y después de la limpieza con los mismos estímulos, recorriendo los ocho estados, y las salidas fueron idénticas en todos los ciclos salvo en HEX5, que cambió por la normalización.

En Quartus la compilación terminó sin errores, con 560 celdas lógicas y un margen de 10.85 ns en el peor caso de tiempos. El diseño usa seis celdas menos y tiene más margen que antes, lo que confirma que lo eliminado no cumplía ninguna función. Los cambios se integraron a `development` con un pull request, y en la propuesta de investigación se actualizaron los resultados de la síntesis y se marcaron como resueltos los pendientes del filtro de entradas y de la normalización del display.